

1^{re} - NSI - FICHE 4 - TYPES ET VALEURS DE BASE

LES BOOLEENS

En 1847, le britannique George BOOLE inventa un formalisme permettant d'écrire des raisonnements logiques : l'algèbre de Boole. La notion même d'informatique n'existait pas à l'époque, même si les calculs étaient déjà automatisés (penser à la Pascaline de 1642).

I. LES BOOLEENS

1. Définition

- ☞ Les valeurs booléennes sont les valeurs 0 et 1 correspondant à l'état d'un bit.
- ☞ Une variable booléenne ne peut donc prendre que l'une ou l'autre de ces valeurs.

En Python, les données de type **bool** sont les données qui ne peuvent prendre qu'une des 2 valeurs suivantes : **True** (vrai pour 1) ou **False** (faux pour 0)

2. Opérateurs de comparaison.

Un booléen peut être le résultat d'une comparaison. Par exemple :

```
a=10
print (a>4)
print (a==2)
```

Les résultats affichés

True 10 > 4
False 10 ≠ 2

Les différents opérateurs de comparaison possibles sont : <, <=, >, >=, ==, !=.

3. Opérateurs booléens

a. L'opérateur non (not)

L'opérateur "**non**" transforme 1 en 0 et 0 en 1.

a	non(a)
1	
0	

ou

a	non(a)
Vrai	
Faux	

Exemple

```
a=4
print (not (a%2))
```

.....
.....

b. L'opérateur et (and)

a et b sont 2 variables booléennes.
L'opérateur **et** appliqué à a et b renvoie

- 1 si les 2 variables valent 1,
- 0 sinon.

a	b	a et b
0	0	0
0	1	0
1	0	0
1	1	1

ou

a	b	a et b
Faux	Faux	Faux
Faux	Vrai	Faux
Vrai	Faux	Faux
Vrai	Vrai	Vrai

Exemple

```
note=15
mention_bien=note>=14 and note<16
print (mention_bien)
```

Le programme affiche

c. L'opérateur ou (or)

a et b sont 2 variables booléennes.

L'opérateur **ou** appliqué à a et b renvoie

- 1 si au moins l'une des 2 variables vaut 1,
- 0 sinon.

Exemple

```
note=18
extreme=note<5 or note>16
print(extreme)
```

Le programme affiche

a	b	a ou b
0	0	0
0	1	1
1	0	1
1	1	1

ou

a	b	a ou b
Faux	Faux	f
f	V	V
V	f	V
V	V	V

d. L'opérateur ou exclusif (xor)

a et b sont 2 variables booléennes.

L'opérateur **ou exclusif** appliqué à a et b renvoie

- 1 si et seulement si exactement une seule des 2 variables booléennes vaut 1
- 0 sinon.

a	b	a xor b
0	0	0
0	1	1
1	0	1
1	1	0

ou

a	b	a xor b
Faux	Faux	f
f	V	V
V	f	V
V	V	f

Remarques :

- En Python, l'opérateur est $\wedge$.
- L'opérateur xor renvoie 1 uniquement si a et b ne sont pas égales.

En Python, l'instruction `not(a==b)` ou `a!=b` est donc équivalente à l'instruction `a  $\wedge$  b`.

II. LES EXPRESSIONS BOOLÉENNES

Une expression booléenne est une expression construite à l'aide de variables booléennes et d'opérateurs booléens. Elle prend pour valeur 0 ou 1.

Exemple : l'expression **(a et b) ou non(c)** est une expression booléenne.

À VOUS DE FAIRE : compléter la table de vérité suivante :

a	b	c	a et b	non(c)	(a et b) ou non(c)
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Remarque : ces opérateurs tirent parti des propriétés suivantes qui permettent d'accélérer le calcul d'expressions booléennes dans certains cas :

- ☛ **Faux et B** est faux quelle que soit la valeur de B;
- ☛ **Vrai ou B** est vrai quelle que soit la valeur de B.

Par exemple, si on a une expression **A et B** et que l'on a calculé que A est faux, le résultat est faux. Il en est de même pour une expression **A ou B** : A est vrai alors le résultat est vrai.

⚠ L'ordre des termes dans une expression booléenne a de l'importance :

- ☞ $x \neq 0$ and $y/x > 0$ est faux si x est nul.
- ☞ $y/x > 0$ and $x \neq 0$ provoque une erreur si x est nul (division par 0).

En résumé :

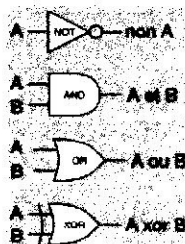
- dans l'expression **A and B and C and...**, la première condition fausse arrête le calcul de l'expression. Et cette expression vaut **False**.
- dans l'expression **A or B or C or...**, la première condition vraie arrête le calcul de l'expression. Et cette expression vaut **True**.

III. LES CIRCUITS LOGIQUES BINAIRES

Les circuits intégrés des ordinateurs sont constitués de portes logiques. Des composants électroniques miniatures, les transistors, peuvent laisser passer ou non un signal électrique.

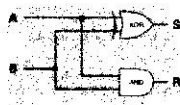
La présence d'un signal électrique sera associée à la valeur logique Vrai ou 1, et son absence à la valeur logique Faux ou 0.

Les portes logiques permettent de combiner ces signaux pour effectuer les opérations de la logique booléenne. Les portes logiques sont représentées conventionnellement par les schémas ci-dessous :



On peut combiner les portes logiques pour construire les composants de base d'un ordinateur.

Dans le schéma ci-dessous, la sortie S est la somme des bits A et B et R la retenue. Ainsi, si A vaut 1 et B vaut 1, la somme est 10 donc S vaut 0 et R vaut 1.



IV. EXERCICES

1. **QCM** Pour chaque question, une seule réponse est exacte parmi celles proposées.

- La réponse à l'instruction `"1==3"` est :
 - une variable numérique
 - une chaîne de caractères
 - un booléen
- On souhaite mettre le résultat de l'opération $9-3-(7-2)*4$ dans une variable résultat. Le type de la variable résultat sera :
 - str
 - int
 - bool
 - float
- Dans un programme Python, si `a="8"` et `b=5`, que retourne l'instruction `a-b` ?
 - "13"
 - 13
 - False
 - TypeError : must be str, not int
- Clément a 100 euros. Chaque semaine, il dépense 10% de ce qu'il a. Il souhaite savoir combien il possèdera après 10 semaines. la classe Python de la variable représentant la somme de Clément est la classe :
 - int
 - float
 - bool
 - str

2. Dans chacun des cas suivants donner la valeur du booléen rep :

- a. $x=-3$ $rep=x**2==9$ *False*
 b. $x,y,z=3,4,5$ $rep=x**2+y**2==z**2$ *True*
 c. $a,b=3,-7$ $rep=a**3>50$ and $b**2<50$ *False*
 d. $a,b=3,-7$ $rep=(a**3>50$ and $b**2<50)$ or $(a**2<10$ and $b**2>10)$ *True.*

3. On donne les quatre fonctions suivantes programmées en Python. À quel opérateur logique correspondent-elles ? Justifiez la réponse.

```
def f(a,b):
    return a*b
```

```
def f(a,b):
    return abs(a-b)
```

```
def f(a,b):
    return a·b·a*b
```

```
def f(a,b):
    return 1·a
```

4. Compléter les tableaux suivants :

a	b	not(a) and b	not(a) or b	not(a) and not(b)	not(a) or not(b)
0	0	<i>F</i>	<i>✓</i>	<i>✓</i>	<i>✓</i>
0	1	<i>✓</i>	<i>✓</i>	<i>F</i>	<i>✓</i>
1	0	<i>F</i>	<i>F</i>	<i>F</i>	<i>✓</i>
1	1	<i>F</i>	<i>✓</i>	<i>F</i>	<i>F</i>

a	b	(a or b) and (a and b)
0	0	<i>F</i>
0	1	<i>F</i>
1	0	<i>F</i>
1	1	<i>✓</i>

a	b	(a or b) or (a and b)
0	0	<i>F</i>
0	1	<i>✓</i>
1	0	<i>✓</i>
1	1	<i>✓</i>

5. Soient a et b deux booléens. On définit la loi Delta telle que : $a \text{ Delta } b = (a \text{ or } b) \text{ and not}(a \text{ and } b)$

a. Compléter la table de la loi Delta suivante :

Delta	0	1
0	<i>F</i>	<i>✓</i>
1	<i>✓</i>	<i>F</i>

b. Que se cache-t-il derrière la loi Delta ? *-> un xor.*

c. Dessiner un circuit logique qui calcule **A xor B** sans utiliser de porte XOR.

6. Vérifier les Lois De Morgan :

a. $\text{not}(A \text{ and } B) == \text{not } A \text{ or not } B$ *True*

b. $\text{not}(A \text{ or } B) == \text{not } A \text{ and not } B$ *False*

V. UNE VIDEO

<https://www.youtube.com/watch?v=r-crdd7eKBac>

